Title: Law of Recursion (bounded, measure-preserving, ergodic, fixed coarse-graining) Author: Placeholder Author Version: 1.0 Date: 2026-02-10 Status: Defendable technical note Abstract: This note specifies a narrow, operational claim about deterministic discrete-time recursion on a bounded state space with a fixed coarse-graining observer. The claim is limited to measurable outcomes: convergence of coarse-grained empirical distributions and conservation of invariants when the boundary operator is inactive. All terms are defined in code-level terms where possible, and predictions are tied to observable quantities in the provided toy model and tests.

0.1 Table of Contents

- Scope and Intent
- Method / Instrument (Theory of Emergence)
- Core Claim
- Operationalization in Code
- Predictions and Falsifiers
- Experiment Results (2026-02-10)
- Relation to Known Frameworks
- Non-applicability / Boundary Conditions
- Limitations
- Reproducibility & Artifacts
- What Would Change My Mind?

0.2 Scope and Intent

This document is a compact, testable technical note. It makes no claims beyond deterministic discrete-time systems with bounded state spaces, fixed coarse-graining, and explicitly stated assumptions. It is not a general theory of recursion in software, biology, or physics.

0.3 Method / Instrument (Theory of Emergence)

This instrument is used once as a reading guide and as a guardrail for scope control.

- FRAME: Define the system, observer model, and temporal scale.
- GRADIENT: List explicit assumptions and failure modes.
- VECTOR: State the precise claim and its operational predictions.
- MOMENT: Clarify what is and is not being claimed.
- DRIFT CHECK: List disallowed extensions and out-of-scope interpretations.

0.4 Core Claim

Law of Recursion (bounded, measure-preserving, ergodic, fixed coarse-graining). For a deterministic recursion on a bounded state space with a fixed

coarse-graining observer, if the iteration map is measure-preserving and ergodic on the relevant domain and the boundary operator is inactive on the trajectory, then repeated iteration yields a unique coarse-grained invariant distribution. Any state function invariant under the iteration map is conserved along the trajectory while the boundary remains inactive.

0.4.1 Assumptions

1. A1. Bounded state space: all states are constrained by a boundary operator `B` (defined in D3) that maps states into a bounded region.
2. A2. Deterministic recursion: a fixed iteration map `F` (defined in D2) is applied at each step, followed by `B`.
3. A3. Fixed coarse-graining: the observer map `O` (defined in D4) from state space to a finite code set is fixed over time.
4. A4. Measure-preserving iteration on the relevant domain. OUT OF SCOPE for generic computation; assumed by construction in the toy model when `B` is inactive.
5. A5. Ergodicity on the relevant domain. OUT OF SCOPE for generic computation; assumed by construction in the toy model for irrational rotation.
6. A6. Boundary inactivity on the trajectory: for the states under analysis, `B` acts as identity (no clipping or projection).
7. A7. Observational noise (if present) is bounded and smaller than the coarse-graining bin width. OUT OF SCOPE in the current codebase (no noise model).

0.4.2 Definitions

1. D1. Recursion: repeated application of B
   $circF$ to an initial state.
2. D2. Iteration map `F`: the deterministic update function; in code this is `_rotate` in `src/recursion_law/model.py`.
3. D3. Boundary operator `B`: the function enforcing boundedness; in code this is `_apply_boundary` in `src/recursion_law/model.py`.
4. D4. Observer/coarse-graining map `O`: the finite-bin mapping from state to observed code; in code this is `coarse_grain` in `src/recursion_law/model.py`.
5. D5. Trajectory: the list of states produced by `iterate(state, params, steps)`.
6. D6. Coarse-grained histogram: the empirical distribution of `O(state)` over a trajectory, computed in `invariant_measure`.
7. D7. Coarse-grained invariant distribution (operational): the histogram produced by `invariant_measure` after a specified number of steps.
8. D8. Uniqueness (operational): two trajectories with different initial states but identical parameters yield histograms within the same numerical tolerance.

9. D9. Conserved quantity: any function $I(s)$ such that $I(F(s)) = I(s)$ on the boundary-inactive domain; in this codebase, radius is the conserved quantity for rotation.
10. D10. Measure-preserving (operational proxy): for the rotation model with boundary inactive, the map is assumed measure-preserving by analytic construction. OUT OF SCOPE for generic verification in code.
11. D11. Ergodic (operational proxy): for the rotation model with irrational $omega/(2$
$pi)$, ergodicity is assumed by analytic construction. OUT OF SCOPE for generic verification in code.

0.5 Operationalization in Code

- State: `(x, y)` floats, `State = Tuple[float, float]` in `src/recursion_law/model.py`.
- `F`: `_rotate(state, omega)` in `src/recursion_law/model.py`.
- `B`: `_apply_boundary(state, boundary_radius)` in `src/recursion_law/model.py`.
- B
$circF$ recursion: `iterate(state, params, steps)` in `src/recursion_law/model.py`.
- `O`: `coarse_grain(state, params)` in `src/recursion_law/model.py`.
- Observables: `invariant_measure(trajectory, params)` returns `radius_mean`, `radius_std`, `bin_l1_to_uniform`.
- Predictions are exercised in `tests/test_law_of_recursion.py` and summarized by `scripts/run_recursion_experiments.py`.

0.6 Predictions and Falsifiers

0.6.1 Predictions

1. P1. Uniformity under irrational rotation: with
$omega/(2$
$pi)$ irrational, `boundary_radius = 1.0`, `bins = K`, and `steps >= 10_000`,

$textttbin_l1_to_uniform$
$le0.15$.
2. P2. Conservation of radius: with r
le
$textttboundary_radius$ and boundary inactive,
$textttradius_std$
$le1e{-10}$ and $|$
$textttradius_mean - r|$
$le1e{-9}$.
3. P3. Non-uniformity under rational rotation: with
$omega = 2$
$pi(p/q)$, `bins = K > q`, and `steps` a multiple of `q`,
$textttbin_l1_to_uniform$
$ge(K - q)/K - 1e{-9}$.

0.6.2 Falsifiers

1. F1. Under P1 conditions,
   $textttbin_l1_to_uniform > 0.15$ after `steps >= 10_000`.
2. F2. Under P2 conditions,
   $textttradius_std > 1e{-}10$ or |
   $textttradius_mean - r| > 1e{-}9$.
3. F3. Under P3 conditions,
   $textttbin_l1_to_uniform < (K - q)/K - 1e{-}9$.

0.6.3 Prediction-to-Falsifier Mapping

Table: Prediction-to-falsifier mapping. | Prediction | Observable | How computed | Threshold / tolerance | Corresponding falsifier | | — | — | — | — | — | | P1 | `bin_l1_to_uniform` | `iterate` then `invariant_measure` | `<= 0.15` at `steps >= 10_000` | F1 | | P2 | `radius_mean`, `radius_std` | `iterate` then `invariant_measure` | `\|radius_mean - r\| <= 1e-9`, `radius_std <= 1e-10` | F2 | | P3 | `bin_l1_to_uniform` | `iterate` then `invariant_measure` | `>= (K - q) / K - 1e-9` | F3 |

0.7 Experiment Results (2026-02-10)

Recorded with `scripts/run_recursion_experiments.py` using `bins = 16`, `boundary_radius = 1.0`, and initial state on the unit circle. `omega = 2 * math.pi * (math.sqrt(2) % 1.0)` for the irrational case and `omega = 2 * math.pi * (1/4)` for the rational case.

Table: Experiment results for irrational vs rational rotation (2026-02-10). | label | steps | bins | radius_std | bin_l1_to_uniform | | — | — | — | — | — | | irrational | 500 | 16 | 3.880e-15 | 0.0245 | | irrational | 1,000 | 16 | 7.265e-15 | 0.0099 | | irrational | 2,000 | 16 | 1.480e-14 | 0.0047 | | irrational | 5,000 | 16 | 3.565e-14 | 0.0025 | | irrational | 10,000 | 16 | 7.084e-14 | 0.0009 | | irrational | 20,000 | 16 | 1.418e-13 | 0.0007 | | rational_1_4 | 500 | 16 | 0.000e+00 | 1.4800 | | rational_1_4 | 1,000 | 16 | 0.000e+00 | 1.4900 | | rational_1_4 | 2,000 | 16 | 0.000e+00 | 1.4950 | | rational_1_4 | 5,000 | 16 | 0.000e+00 | 1.4980 | | rational_1_4 | 10,000 | 16 | 0.000e+00 | 1.4990 | | rational_1_4 | 20,000 | 16 | 0.000e+00 | 1.4995 |

Plot output is written to `scripts/recursion_experiments.png` when matplotlib is available. The plot is not embedded here; see Reproducibility & Artifacts.

0.8 Relation to Known Frameworks

- Dynamical systems: the claim concerns fixed points of coarse-grained distributions, not necessarily point attractors in state space.
- Coarse-graining and renormalization: `coarse_grain` is an observer map, not a physical renormalization flow.

- Information conservation: the only conservation asserted is of explicit invariants under `F` when `B` is inactive.

0.9 Non-applicability / Boundary Conditions

The law makes no prediction under any of the following conditions.

1. N1. The boundary operator is active for the states under analysis (clipping or projection occurs).
2. N2. The iteration map is not measure-preserving on the relevant domain.
3. N3. The dynamics are not ergodic on the relevant domain.
4. N4. The observer map `O` changes over time or is not fixed.
5. N5. The state space is unbounded or the recursion is continuous-time without a discretization.
6. N6. Stochastic noise is present without a bounded noise model and tolerance.

0.10 Limitations

1. L1. Measure preservation and ergodicity are assumed by analytic construction in the toy model, not verified by a general procedure in code.
2. L2. Thresholds and tolerances are specific to the current model, binning scheme, and step counts; they do not generalize without recalibration.
3. L3. The observer model is angular binning only; other coarse-grainings may yield different invariant distributions.
4. L4. Stochasticity and continuous-time dynamics are not modeled.

0.11 Reproducibility & Artifacts

0.11.1 Commands

1. Run experiments and generate the plot:

```
python scripts/run_recursion_experiments.py
```

Expected output pattern includes a table with columns `label`, `steps`, `bins`, `radius_std`, `bin_l1`. The numeric values should be close to the Experiment Results above. The script also writes `scripts/recursion_experiments.png` when matplotlib is available.

2. Run tests:

```
python -m pytest tests/test_law_of_recursion.py
```

Expected output includes all tests passing (currently 5 tests).

3. Build the PDF:

```
pandoc docs/LAW_OF_RECURSION_FINAL.md -o docs/LAW_OF_RECURSION_FINAL.pdf --pdf-engine=pdflat
```

0.11.2 Hardware, Seeds, and Numerical Tolerance

- Determinism: `scripts/run_recursion_experiments.py` is deterministic for fixed parameters.
- Randomness: `tests/test_law_of_recursion.py` uses `random.seed(0)` in `test_randomized_radius_invariance`.
- Numerical tolerance: tolerances are set to `1e-9` or `1e-10` for radius statistics and `0.15` for L1 uniformity. Floating-point differences across platforms should remain within these bounds for the tested step counts.
- Headless plotting: if matplotlib is available but cannot open a display, set `MPLBACKEND=Agg` and `MPLCONFIGDIR` to a writable directory.

0.11.3 Artifacts (repo paths)

- `docs/LAW_OF_RECURSION_FINAL.md`
- `docs/LAW_OF_RECURSION_FINAL.pdf`
- `src/recursion_law/model.py`
- `tests/test_law_of_recursion.py`
- `scripts/run_recursion_experiments.py`
- `scripts/recursion_experiments.png`

0.12 What Would Change My Mind?

1. A reproducible counterexample within the stated domain where a measure-preserving, ergodic recursion fails to yield a unique coarse-grained invariant distribution (operationally, histograms diverge beyond stated tolerances).
2. A reproducible case within the stated domain where a conserved quantity under `F` drifts while the boundary is inactive.
3. A reproducible case within the stated domain where long-run coarse-grained distributions depend on initial conditions despite the ergodicity assumption.